

گزارش پروژه برنامه‌سازی وب

۱. مقدمه و تبیین معماری کلان سیستم

این سند نمایانگر گزارش نهایی، جامع و تحلیلی از فرآیند مهندسی، طراحی و پیاده‌سازی سامانه استریم موسیقی (پلتفرمی با کارکرد مشابه Spotify) است که در مخزن تعیین‌شده توسعه یافته است. معماری این سیستم بر پایه الگوی کلاينت-سرور با جداسازی کامل لایه رابط کاربری (Frontend) و منطق تجاری سرور (Backend) بنا شده است. این رویکرد نه تنها مقیاس‌پذیری سامانه را در برابر بار ترافیکی بالا تضمین می‌کند، بلکه امکان توسعه مستقل هر لایه را برای تیم مهندسی فراهم می‌آورد. بر اساس مستندات و نیازمندی‌های پروژه، این پلتفرم برای خدمت‌رسانی به چهار دسته مجزا از کاربران شامل شنوندگان عادی، هنرمندان، تیم پشتیبانی و مدیر ارشد سامانه طراحی شده است^۱.

مدیریت سطوح دسترسی و سیاست‌های قیمت‌گذاری، هسته مرکزی منطق تجاری این سیستم را تشکیل می‌دهد. سیستم اشتراک کاربران در سه سطح پایه (رایگان)، نقره‌ای و طلایی پیاده‌سازی شده است که هر یک محدودیت‌ها و امتیازات مشخصی را به همراه دارند. در معماری بک‌اند، این محدودیت‌ها به صورت میان‌افزارها (Middlewares) و مجوزهای اختصاصی (Custom Permissions) تعریف شده‌اند تا از هرگونه دسترسی غیرمجاز به منابع جلوگیری شود. جدول زیر، معماری سطوح دسترسی و مزایای هر اشتراک را که مستقیماً در لایه پایگاه داده و منطق برنامه پیاده‌سازی شده است، نشان می‌دهد.

قابلیت‌های سامانه	اشتراک پایه (رایگان)	اشتراک نقره‌ای	اشتراک طلایی
محدودیت تعداد استریم در روز	حداکثر ۶۰ قطعه	نامحدود (∞)	نامحدود (∞)
محدودیت تعداد پلی‌لیست‌ها	حداکثر ۶ عدد	حداکثر ۱۰۰ عدد	نامحدود (∞)
افزودن و تغییر عکس نمایه	مسدود	فعال	فعال
دانلود آهنگ برای پخش آفلاین	مسدود	فعال	فعال
دسترسی زودهنگام به آثار جدید	مسدود	مسدود	فعال
مشاهده آمارهای پیشرفته شنوندگان	مسدود	مسدود	فعال

همچنین سیستم قیمت‌گذاری به صورت کاملاً پویا (Dynamic Pricing) طراحی شده است، به گونه‌ای که مدیر سامانه می‌تواند از طریق داشبورد مدیریت، بدون نیاز به هیچ‌گونه تغییر در سورس‌کد یا راه‌اندازی مجدد سرور، هزینه‌های اشتراک را تنظیم نماید^۱. این نیازمندی مستلزم طراحی یک مدل مستقل برای تنظیمات سیستمی در پایگاه داده بود تا همواره آخرین پیکربندی مالی در حافظه پنهان (Cache) سرور قرار گیرد. در ادامه این گزارش، به تحلیل عمیق تکتک بخش‌های خواسته‌شده شامل توزیع وظایف، قواعد مهندسی نرم‌افزار، ساختار دایرکتوری‌ها، شاخص‌های نگهداری‌پذیری، معماری داده و در نهایت نقش ابزارهای هوش مصنوعی در توسعه این

۲. نقش و کارهای کلی هر عضو در هر یک از دو فاز

توسعه این سامانه بر اساس متدولوژی توسعه چابک (Agile) و در قالب دو فاز اصلی برنامه‌ریزی و اجرا گردید. فاز اول به صورت کامل به ایجاد بخش فرانت‌اند (با استفاده از داده‌های ماک در Local Storage) و فاز دوم به توسعه بک‌اند و اتصال کامل این دو لایه اختصاص داشت^۱. با توجه به الزام مشارکت تمامی اعضا در تمام مراحل پروژه، تقسیم کار به گونه‌ای مهندسی شد که هر سه عضو تیم درک عمیقی از جریان داده (Data Flow) از پایگاه داده تا رابط کاربری پیدا کنند.

در فاز نخست، عضو اول تیم مسئولیت خطیر معماری پایه فرانت‌اند و پیاده‌سازی پیچیده‌ترین جزء تعاملی سیستم، یعنی پخش‌کننده موسیقی (Music Player) را بر عهده گرفت. این پخش‌کننده نیازمند مدیریت وضعیت (State Management) بسیار دقیقی بود تا نوار پیشرفت (Progress Bar)، کنترل‌های پخش، شافل و تکرار به صورت کاملاً هماهنگ عمل کنند^۱. این عضو همچنین بخش فعالیت امتیازی مربوط به پخش‌کننده پیشرفته (تعیین کیفیت، افکت Crossfade و هماهنگی رنگ قالب کاور با ویدجت‌ها) را توسعه داد^۱. عضو دوم تیم، تمرکز خود را بر روی طراحی و پیاده‌سازی صفحات نمایه کاربران و هنرمندان، سیستم اعلانات با قابلیت تفکیک پیام‌های خوانده‌نشده، و همچنین داشبوردهای پیچیده تیم پشتیبانی و مدیر سامانه معطوف کرد. توسعه رابط کاربری بخش حساس‌ری مالی و پنل کنترل قیمت‌ها که نیازمند دقت بالا در نمایش داده‌های حساس بود، توسط این عضو انجام شد. عضو سوم، توسعه ویدئو اصلی سامانه (صفحه خانه)، موتور جستجوی یکپارچه، فیلترهای مرتب‌سازی بر اساس آمار شنوندگان، و مدیریت لیست‌های پخش (Playlists) را بر عهده داشت. ایشان همچنین مکانیزم ذخیره‌سازی داده‌های ماک در مرورگر را برای تست‌پذیری فاز اول مهندسی کرد^۱.

با ورود به فاز دوم، تمرکز تیم به سمت معماری سرور، پایگاه داده و ادغام سرویس‌ها تغییر یافت. عضو اول تیم، مدل‌سازی پایه در فریم‌ورک Django Rest، پی‌کردنی پایگاه داده و توسعه اندپوینت‌های عملیاتی (CRUD) را با استفاده از ابزارهای Django Rest Framework آغاز کرد^۱. داکرایز کردن هر دو پروژه فرانت‌اند و بک‌اند و نگارش فایل‌های docker-compose.yml برای اجرای یکپارچه محیط توسعه نیز توسط ایشان انجام پذیرفت. عضو دوم، پیاده‌سازی منطق پیچیده اشتراک‌ها، مدیریت سطوح دسترسی (RBAC) و ادغام سیستم با درگاه‌های پرداخت آزمایشی (مانند Zarinpal و PayPing) را بر عهده گرفت^۱. یکی از مهم‌ترین دستاوردهای این عضو، انتقال کامل بار محاسباتی مربوط به گزارش‌گیری و حساس‌ری مالی به لایه بک‌اند بود تا از افت کارایی مرورگر کاربران جلوگیری شود. عضو سوم نیز بر روی مدیریت امن فایل‌های رسانه‌ای (آپلود فایل‌های FLAC/WAV/MP3 و تصاویر) و معماری همگام‌سازی پیش‌فرض‌های کاربران بین دستگاه‌های مختلف متمرکز شد. توسعه زیرساخت بلادرنگ (Real-time) برای پیاده‌سازی بخش امتیازی "شنیدار گروهی" (Group Listening) با استفاده از WebSockets نیز از دیگر دستاوردهای مهم عضو سوم در فاز دوم بود^۱.

نام	مسئولیت‌های کلیدی در فاز اول (فرانت‌اند)	مسئولیت‌های کلیدی در فاز دوم (بک‌اند و ادغام)
ماهان معصوم زاده	معماری پایه React، پخش‌کننده موسیقی پیشرفته، مدیریت State، قابلیت Crossfade	طراحی مدل‌های پایه Django، اندپوینت‌های CRUD، داکرایز کردن پروژه
امیر علی منصوریان	داشبوردهای مدیریتی و حساس‌ری، نمایه‌های کاربری، سیستم اعلانات تعاملی	منطق اشتراک‌ها، ادغام درگاه پرداخت، کوئری‌های تجمیعی پایگاه داده
امیر علی خدایی	صفحات کشف موسیقی (خانه، آلبوم‌ها)، جستجو و فیلتر، سیستم Mock محلی	مدیریت بارگذاری فایل‌های رسانه، همگام‌سازی پیش‌فرض‌ها

۳. قوانین توسعه و فرآیند آن

توسعه نرم‌افزاری در ابعاد این پروژه، بدون استقرار و پایبندی به مجموعه‌ای از قوانین مهندسی و استانداردهای تیمی، به سرعت منجر به بروز خطاهای یکپارچه‌سازی و بدهی فنی (Technical Debt) می‌شود. از این رو، پیش از نگارش اولین خط کد، تیم توسعه یک میثاق‌نامه فنی را تدوین و در مستندات داخلی مخزن ثبت نمود.

نخستین قانون، پایبندی بی‌قید و شرط به استانداردهای نام‌گذاری (Naming Conventions) بود. در لایه بک‌اند که با زبان Python توسعه یافت، استاندارد PEP8 به عنوان مرجع نهایی انتخاب شد. تمامی متغیرها، توابع و نام فیلدهای پایگاه داده به فرمت snake_case نوشته شدند تا خوانایی کد در بالاترین سطح حفظ شود. در مقابل، نام کلاس‌ها شامل مدل‌ها، سریالایزرها و کلاس‌های ویو (View) از فرمت PascalCase پیروی کردند. در لایه فرانت‌اند که مبتنی بر اکوسیستم JavaScript/React بود، قراردادهای متفاوتی اعمال شد. نام تمامی فایل‌های کامپوننت و توابع سازنده آن‌ها با PascalCase تعریف شدند، در حالی که توابع کمکی، هوک‌های سفارشی (Custom Hooks) و متغیرهای محلی به صورت camelCase نوشته شدند. مقادیر ثابت و متغیرهای محیطی نیز برای تمایز بصری با فرمت UPPER_SNAKE_CASE پیاده‌سازی شدند.

استراتژی مدیریت انشعاب‌ها (Branching Strategy) بر اساس مدل استاندارد Git Flow طراحی گردید. شاخه main به عنوان محیط پایدار و عملیاتی (Production) ایزوله شد و هیچ‌کس اجازه Commit مستقیم روی آن را نداشت. تمامی یکپارچه‌سازی‌ها در شاخه develop انجام می‌گرفت. برای هر ویژگی جدید (مانند اتصال به درگاه پرداخت یا ساخت پلیر)، یک شاخه با پیشوند /feature ایجاد می‌شد. پس از اتمام توسعه یک ویژگی، کد مربوطه از طریق یک Pull Request (PR) به شاخه develop ارسال می‌گردید. قانون سخت‌گیرانه‌ای وضع شد که هیچ PRی بدون بازبینی کد (Code Review) توسط حداقل یک عضو دیگر تیم، اجازه ادغام (Merge) نداشت. این فرآیند باعث شد تا خطاهای منطقی پیش از ورود به کدبیس اصلی شناسایی و رفع شوند. قوانین مربوط به پیام‌های کامیت (Commit Messages) نیز بر اساس ساختار Conventional Commits پیاده‌سازی شد. این ساختار تیم را ملزم می‌کرد تا ماهیت هر تغییر را در ابتدای پیام مشخص کنند؛ مانند استفاده از پیشوند feat: برای افزودن قابلیت‌های جدید، fix: برای رفع باگ‌های گزارش‌شده در سیستم تیکتینگ، refactor: برای بازنویسی کدهایی که نیاز به بهبود ساختار داشتند (بدون تغییر در عملکرد)، و docs: برای به‌روزرسانی فایل‌های مستندات^۱. این نظم در تاریخچه Git، فرآیند دیباگ کردن و یافتن ریشه خطاها در طول چرخه توسعه را به شدت تسریع بخشید.

۴. ساختار هر یک از دو پروژه و توجیه آن

ساختار دایرکتوری‌ها و معماری پوشه‌های یک نرم‌افزار، نمایانگر معماری فکری تیم توسعه‌دهنده است. در این سامانه، رویکرد تفکیک دغدغه‌ها (Separation of Concerns) به عنوان اصل اساسی در چیدمان فایل‌ها در هر دو لایه فرانت‌اند و بک‌اند مورد توجه قرار گرفت تا سیستم در برابر تغییرات آینده مقاومت بالایی داشته باشد^۱.

در لایه فرانت‌اند، ساختار پروژه بر اساس الگوی طراحی اتمی (Atomic Design) و معماری مبتنی بر ویژگی (Feature-based) ساماندهی شد. پوشه src/components به زیرشاخه‌هایی تقسیم شد که هر کدام سطح پیچیدگی متفاوتی از رابط کاربری را در بر می‌گرفتند. دکمه‌ها، ورودی‌های متن و آیکون‌ها به عنوان اجزای پایه توسعه یافتند. این اجزا در کامپوننت‌های پیچیده‌تری مانند کارت‌های تک‌آهنگ یا فرم‌های ثبت‌نام هنرمندان ترکیب شدند. صفحات اصلی سامانه که وظیفه مدیریت مسیرها (Routing) را بر عهده داشتند در پوشه مجزایی قرار گرفتند. یکی از مهم‌ترین تصمیمات معماری در این لایه، ایجاد پوشه src/services بود. تمامی فراخوانی‌های شبکه و تعامل با API‌های بک‌اند در این پوشه متمرکز شدند. این تصمیم، کامپوننت‌های رابط کاربری را از وابستگی مستقیم به کتابخانه‌های دریافت داده (مانند Axios) رها ساخت. در صورت تغییر ساختار پاسخ‌های سرور، صرفاً فایل‌های سرویس نیاز به اصلاح داشتند و هیچ‌یک از کامپوننت‌های نمایشی دستخوش تغییر نمی‌شدند. همچنین، منطق پیچیده پخش‌کننده موسیقی و مدیریت صف پخش در پوشه src/store و با استفاده از ابزارهای مدیریت وضعیت سراسری استقرار یافت تا از رندر شدن مجدد و غیرضروری صفحات جلوگیری شود^۱.

در لایه بک‌اند، معماری مبتنی بر برنامه‌های مستقل (Multi-app Architecture) فریم‌ورک جنگو به بهترین شکل ممکن

پیاده‌سازی شد. به جای قرار دادن تمام مدل‌ها و منطق تجاری در یک اپلیکیشن واحد و حجیم، سیستم به چندین اپلیکیشن کوچک با مسئولیت‌های کاملاً مشخص تقسیم گردید. پوشه **core** وظیفه پیکربندی‌های اصلی، مدیریت متغیرهای محیطی و تنظیمات امنیتی را بر عهده گرفت. اپلیکیشن **users** به طور انحصاری برای مدیریت مدل کاربر سفارشی، احراز هویت مبتنی بر توکن، و نگهداری اطلاعات پروفایل هنرمندان و شنوندگان طراحی شد. اپلیکیشن **music** به عنوان قلب تپنده سامانه، مدل‌های مربوط به آثار صوتی، آلبوم‌ها، ژانرها و متادیتای مرتبط با آن‌ها را در خود جای داد. سیستم حسابرسی، مدیریت اشتراک‌های سه‌گانه و یکپارچگی با درگاه‌های پرداخت در اپلیکیشن مستقلی به نام **finance** متمرکز شد. در نهایت، اپلیکیشن **support** برای مدیریت سیستم تیکتینگ و اعتبارسنجی هنرمندان توسعه یافت^۱. این تفکیک دقیق مرزها، توجیه بسیار روشنی دارد: کاهش در هم‌تنیدگی (Coupling) و افزایش انسجام (Cohesion). با این ساختار، هر اپلیکیشن می‌تواند به صورت مستقل توسعه یافته، تست شده و حتی در آینده در معماری مایکروسرویس‌ها به سرویس‌های جداگانه‌ای تبدیل شود.

۵. توجیه نگهداری‌پذیری کد (Maintainability)

شاخص نگهداری‌پذیری کد یکی از حیاتی‌ترین معیارهای ارزیابی در مستندات پروژه بود و صرفاً کارکرد صحیح سیستم برای کسب نمره کامل کفایت نمی‌کرد^۱. تیم توسعه برای تضمین این ویژگی، استانداردهای توسعه نرم‌افزار تمیز (Clean Code) و الگوهای طراحی (Design Patterns) را به صورت سیستماتیک در تمام مراحل پیاده‌سازی اعمال نمود. رعایت اصل عدم تکرار (DRY - Don't Repeat Yourself) یکی از پایه‌های اصلی در توجیه نگهداری‌پذیری این سیستم است. در لایه بک‌اند، استفاده هوشمندانه از ابزارهای (Django Rest Framework (DRF نقش بسزایی در کاهش حجم کدهای زائد داشت. به جای نوشتن ویوهای طولانی برای اعتبارسنجی و ذخیره داده‌ها، از ModelViewSet‌ها استفاده شد که عملیات استاندارد CRUD را به صورت خودکار و امن هندل می‌کنند. این رویکرد تضمین کرد که اندپوینت‌ها دقیقاً به اندازه نیاز طراحی شوند؛ به عنوان مثال، اگر به روزرسانی یک منبع خاص در منطق تجاری سیستم توجیهی نداشت، متدهای PUT یا PATCH به طور کلی از اندپوینت مربوطه حذف شدند^۱. در فرانت‌اند نیز، توابع پیچیده‌ای نظیر محاسبه زمان پخش آهنگ یا اعتبارسنجی فرم‌ها به صورت توابع کمکی (Utility Functions) استخراج شدند تا در بخش‌های مختلف برنامه به سادگی فراخوانی شوند. اصل مسئولیت واحد (Single Responsibility Principle) از مجموعه اصول SOLID به شدت در پروژه دنبال شد. هیچ کلاس یا تابعی بیش از یک وظیفه مشخص را بر عهده ندارد. برای مثال، فرآیند محاسبه پاداش مالی هنرمندان که تابعی از تعداد شنوندگان منحصر به فرد و حجم استریم‌هاست، به صورت یک سرویس مجزا در لایه بک‌اند طراحی شد. این منطق به هیچ وجه در ویوهای پاسخ‌دهنده به درخواست‌های HTTP ادغام نگردید^۱. نتیجه این جداسازی آن است که اگر در آینده فرمول محاسبه پاداش هنرمندان تغییر کند، تیم توسعه صرفاً کلاس محاسباتی مربوطه را ویرایش می‌کند و هیچ خطری پایداری سایر بخش‌های سیستم را تهدید نخواهد کرد. تزریق وابستگی (Dependency Injection) و استفاده از کلاس‌های انتزاعی (Abstract Classes) استراتژی دیگری برای حفظ انعطاف‌پذیری سیستم بود. ماژول درگاه پرداخت به گونه‌ای طراحی شد که سیستم هسته، وابستگی مستقیمی به API‌های خاص سرویس‌دهندگانی نظیر Zarinpal یا PayPing نداشته باشد. یک کلاس رابط یکپارچه تعریف شد و سرویس‌دهنده‌های مختلف به عنوان پیاده‌سازی‌های این رابط توسعه یافتند^۱. بدین ترتیب، تغییر درگاه پرداخت در محیط عملیاتی تنها نیازمند تغییر یک متغیر در تنظیمات است و هیچ خط کدی نیاز به بازنویسی نخواهد داشت. همچنین، نوشتن تست‌های جامع و پوشا (Coverage) شامل حداقل ۱۰ تست برای فرانت‌اند و ۱۵ تست برای بک‌اند، پایداری کدها را در برابر تغییرات و ریفکتورهای آینده (Refactoring) تضمین نمود^۱.

۶. مدل‌های موجود در بک‌اند، روابط آن‌ها و توجیه وجودشان

طراحی ساختار داده و معماری مدل‌های بک‌اند، یکی از حساس‌ترین بخش‌های توسعه بود که نیازمند تسلط بر مفاهیم برنامه‌سازی پیشرفته و پایگاه داده رابطه‌ای بود^۱. مدل‌های طراحی‌شده به گونه‌ای بهینه‌سازی شده‌اند که کمترین میزان افزونگی داده را داشته باشند و کوری‌های پیچیده تحلیلی را با سرعت بالا پردازش کنند. در ادامه ساختار و توجیه مدل‌های کلیدی سیستم تشریح می‌گردد:

۶.۱. سیستم کاربران و نمایه‌ها

موجودیت اصلی سیستم، مدل **User** است که با اثربری و سفارشی‌سازی مدل پایه جنگو پیاده‌سازی شد. این مدل دارای فیلدهای اساسی نظیر ایمیل، رمز عبور، نام نمایشی، تاریخ تولد، و یک فیلد کلیدی به نام **Role** (نقش کاربر) است. نقش‌ها شامل شنونده، هنرمند، پشتیبان و مدیر سامانه هستند. به منظور جداسازی اطلاعات تخصصی، مدل **ArtistProfile** با یک رابطه یک‌به‌یک

(One-to-One) به مدل کاربر متصل گردید. این مدل شامل اطلاعاتی نظیر بیوگرافی، وضعیت تأیید حساب (در انتظار، تأیید شده، رد شده) و موجودی قابل تسویه است^۱. **توجه ساختاری:** اگر اطلاعات بیوگرافی و وضعیت تأیید در همان مدل User قرار می‌گرفت، جدول کاربران با فیلدهای خالی متعددی برای شنوندگان عادی مواجه می‌شد. جداسازی پروفایل هنرمند، پایگاه داده را نرمال‌سازی کرده و امنیت داده‌های مالی را افزایش داده است.

۶.۲. معماری سطوح دسترسی

مدل Subscription برای مدیریت طرح‌های مالی کاربران ایجاد شد. این مدل با یک رابطه یک‌به‌یک به کاربر متصل است و شامل فیلدهایی برای تعیین نوع اشتراک (پایه، نقره‌ای، طلایی)، تاریخ شروع و تاریخ انقضا می‌باشد. **توجه ساختاری:** جدا کردن مدل اشتراک از پروفایل کاربر امکان حفظ تاریخچه اشتراک‌ها و تغییر استراتژی‌های فروش در آینده (مانند فروش بسته‌های چندماهه) را بدون دستکاری اطلاعات هویتی کاربر فراهم می‌سازد^۱.

۶.۳. معماری محتوای صوتی و متاداده‌ها

هسته استریم موسیقی بر پایه سه مدل Album، Song و Genre شکل گرفته است. مدل Album شامل عنوان، کاور، تاریخ انتشار و یک کلید خارجی (Foreign Key) به هنرمند سازنده است. مدل Song نمایانگر یک قطعه موسیقی است و شامل فایل صوتی اصلی، عنوان، متن ترانه (Lyrics) و مدت زمان پخش است^۱. رابطه Song با Album به صورت یک کلید خارجی تعریف شده است که می‌تواند مقدار تهی (Null) بپذیرد؛ این تصمیم معماری به سیستم اجازه می‌دهد تا از انتشار "تک‌آهنگ‌ها" که به هیچ آلبومی وابسته نیستند پشتیبانی کند^۱. علاوه بر این، یک رابطه چندبه‌چند (Many-to-Many) تحت عنوان Collaborators در مدل آهنگ تعبیه شده است تا سیستم بتواند هنرمندان همکار در یک قطعه را به درستی شناسایی و حقوق مادی آن‌ها را محاسبه کند.

۶.۴. مدل تحلیلی لاگ استریم و حسابرسی مالی

مدل StreamLog برای ثبت دقیق رفتار شنیداری کاربران طراحی شد. هر بار که کاربری یک قطعه موسیقی را پخش می‌کند، یک رکورد شامل کلید خارجی به کاربر، کلید خارجی به آهنگ و زمان دقیق پخش در این جدول ثبت می‌شود. **توجه ساختاری:** وجود این مدل برای پاسخگویی به دو نیازمندی اساسی پروژه الزامی بود. نخست، اعمال محدودیت روزانه برای کاربران دارای اشتراک پایه (حداکثر ۶۰ استریم در روز) که با شمردن رکوردهای این جدول در ۲۴ ساعت گذشته انجام می‌شود. دوم، محاسبه پاداش مالی هنرمندان؛ سیستم حسابرسی ماهانه با اجرای یک کوئری تجمیعی روی این جدول، تعداد کل استریم‌ها و تعداد شنوندگان منحصر به فرد (Unique Listeners) هر قطعه را استخراج کرده و وارد فرمول پاداش‌دهی می‌کند^۱.

$$(Bonus_Multiplier \times Unique_Listeners) + (Base_Rate \times Total_Streams) = Total_Revenue$$

طراحی این مدل اجازه داد تا تمامی محاسبات سنگین و گزارش‌گیری‌ها در لایه پایگاه داده (توسط متدهای annotate و aggregate در جنگو) انجام پذیرد و هیچ‌گونه بار محاسباتی و پردازشی به فرانت‌اند منتقل نگردد^۱.

۶.۵. سیستم پشتیبانی

مدل‌های Ticket و TicketMessage برای پاسخگویی به نیازمندی داشبورد تیم پشتیبانی طراحی شدند. مدل Ticket حاوی موضوع، فرستنده و وضعیت است، در حالی که TicketMessage دارای کلید خارجی به تیکت مادر بوده و محتوای پیام‌ها را نگهداری می‌کند. این طراحی ساختار درختی و چت‌بکس‌مانند مورد نیاز در مستندات را به درستی پشتیبانی می‌کند^۱.

۷. نقش هوش مصنوعی در توسعه

توسعه سامانه‌ای با این سطح از پیچیدگی در یک بازه زمانی محدود، نیازمند بهرهوری بالا بود. در این راستا، مدل‌های زبانی بزرگ (LLMs) و ابزارهای هوش مصنوعی نقش یک هم‌تیمی و مشاور فنی را ایفا کردند. استفاده از این تکنولوژی به جای جایگزینی تفکر مهندسی، در جهت تسریع کارهای زمان‌بر و تولید ساختارهای اولیه (Boilerplates) متمرکز گردید^۱. یکی از مهم‌ترین کاربردهای هوش مصنوعی در فاز اول، تولید داده‌های آزمایشی (Mock Data) بود. برای ایجاد حس یک پلتفرم واقعی موسیقی پیش از آماده شدن بک‌اند، از هوش مصنوعی برای تولید ده‌ها رکورد شامل نام خوانندگان فرضی، آلبوم‌ها، فایل‌های

صوتی آزمایشی و متادیتای مرتبط با آن‌ها استفاده شد. این داده‌ها از طریق یک اسکریپت تولید شده توسط AI در Local Storage مرورگر تزریق شدند¹. علاوه بر این، در توسعه بخش‌های پیچیده نظیر رابط کاربری و اکشن‌گرا و انیمیشن‌های نوار پخش، هوش مصنوعی با ارائه راهکارهای مبتنی بر Tailwind CSS به تیم کمک شایانی نمود.

در بخش فعالیت امتیازی، یکی از چالش‌برانگیزترین نیازمندی‌ها، پیاده‌سازی سیستم پیشنهاددهنده آهنگ (Recommendation System) بود که نیازمند خروجی‌های منطقی و غیر تصادفی بود¹. هوش مصنوعی در اینجا به عنوان یک معماری داده عمل کرد و تیم را در طراحی یک الگوریتم فیلترینگ محتوایی (Content-based Filtering) راهنمایی نمود. با کمک AI، منطقی طراحی شد که شباهت ژانر، هنرمند و تاریخچه پخش کاربر را به عنوان بردار ویژگی‌ها در نظر گرفته و نزدیک‌ترین آهنگ‌ها را پیشنهاد می‌دهد.

۷.۱. نمونه‌ای از کد هوش مصنوعی در فاز اول (فرانت‌اند)

پیاده‌سازی افکت Crossfade (ادغام ۵ ثانیه انتهای آهنگ فعلی با ۵ ثانیه ابتدای آهنگ بعدی) نیازمند تسلط بر مفاهیم Web Audio API بود¹. هوش مصنوعی با تولید قطعه کد زیر، منطق ریاضی و زمان‌بندی تغییرات دامنه صدا (Gain) را در اختیار تیم قرار داد که سپس بهینه‌سازی و در بستر React یکپارچه گردید:

JavaScript

```
function applyCrossfade(currentAudioNode, nextAudioNode, audioContext, fadeDuration = 5.0) {  
  ; const currentTime = audioContext.currentTime
```

```
// تنظیم کاهش تدریجی صدای آهنگ فعلی
```

```
currentAudioNode.gain.setValueAtTime(1.0, currentTime);  
;( currentAudioNode.gain.linearRampToValueAtTime(0.0, currentTime + fadeDuration
```

```
// تنظیم افزایش تدریجی صدای آهنگ بعدی
```

```
nextAudioNode.gain.setValueAtTime(0.0, currentTime);  
;( nextAudioNode.gain.linearRampToValueAtTime(1.0, currentTime + fadeDuration
```

```
// توقف آهنگ فعلی پس از اتمام Crossfade
```

```
setTimeout(() => {  
  currentAudioNode.source.stop();  
}, fadeDuration * 1000  
}
```

۷.۲. نمونه‌ای از کد هوش مصنوعی در فاز دوم (بک‌اند)

در داشبورد مدیریت، استخراج تعداد شنوندگان منحصربه‌فرد و کل استریم‌های ماهانه هنرمندان به صورت بهینه و بدون انتقال بار پردازشی به فرانت‌اند، نیازمند کوئری‌های پیشرفته ORM بود¹. هوش مصنوعی راهکار استفاده از annotate را به شکل زیر پیشنهاد داد که مستقیماً در لایه پایگاه داده اجرا می‌شود:

```

from django.db.models import Count, Q
from django.utils import timezone
from datetime import timedelta
def get_artist_financial_report(artist_user):
    last_month = timezone.now() - timedelta(days=30)
    report = ArtistProfile.objects.filter(user=artist_user).annotate(
        total_streams=Count(
            'user_song_primary_streamlog',
            filter=Q(user_song_primary_streamlog__timestamp__gte=last_month)
        ),
        unique_listeners=Count(
            'user_song_primary_streamlog_listener',
            filter=Q(user_song_primary_streamlog__timestamp__gte=last_month),
            distinct=True
        )
    ).values('total_streams', 'unique_listeners')
    return report

```

۷.۳. تحلیل ضعف‌ها و قوت‌های هوش مصنوعی در فرآیند توسعه

نقاط قوت هوش مصنوعی در این پروژه غیرقابل انکار بود. در لایه فرانت‌اند، درک عمیق مدل‌های زبانی از اکوسیستم React و هوک‌ها، فرآیند نوشتن کدهای مدیریت وضعیت را بسیار تسریع کرد. در لایه بک‌اند، توانایی AI در نوشتن عبارات باقاعده (Regex) پیچیده برای اعتبارسنجی فرمت فایل‌های صوتی و رمزهای عبور، و همچنین تولید سریع کلاس‌های تست واحد (Unit Tests)، به تیم کمک کرد تا زمان بیشتری را به حل چالش‌های منطقی اختصاص دهد^۱.

با این حال، هوش مصنوعی دارای ضعف‌ها و خطاهای قابل توجهی نیز بود که نیازمند نظارت مهندسی دقیقی بود. پدیده توهم (Hallucination) یکی از بزرگترین چالش‌ها محسوب می‌شد. در طول توسعه فعالیت امتیازی مربوط به استخراج رنگ قالب کاور آهنگ^۱، هوش مصنوعی مکرراً استفاده از کتابخانه‌هایی را پیشنهاد می‌داد که مدت‌ها پیش منسوخ شده بودند یا با نسخه فعلی Next.js تداخل شدیدی داشتند. این مسئله باعث اتلاف وقت تیم برای دیباگ کردن کدهای نامعتبر شد. علاوه بر این، در لایه بک‌اند و در زمان طراحی کونری‌های پیچیده تو در تو (مانند دریافت یک لیست پخش شامل تمام آهنگ‌ها، و هر آهنگ شامل تمام هنرمندان همکار)، کدهای تولید شده توسط AI فاقد بهینه‌سازی‌های لازم مانند استفاده صحیح از prefetch_related بودند. اجرای کورکورانه این کدها منجر به بروز مشکل فاجعه‌بار N+1 Query در پایگاه داده می‌شد که تیم توسعه مجبور به بازنویسی کامل این بخش‌ها به صورت دستی گردید.